Compte rendu — Atelier DevOps CI/CD
Projet `calculatrice-cicd`

Younes BASIR
UPF — DevOps

10 octobre 2026

1 Introduction

Dans le cadre de l'atelier DevOps, j'ai mis en place une chaîne *CI/CD* (intégration et déploiement continus) pour une petite application Python nommée `calculatrice-cicd`. L'objectif était de vérifier automatiquement le code à chaque modification, puis de simuler un déploiement lorsque tout est correct.

J'ai utilisé **pytest** pour les tests, **flake8** pour le style et **GitHub Actions** pour la pipeline. Le dépôt est hébergé sur GitHub ; chaque `push` et chaque `pull request` vers la branche `main` déclenche les jobs définis dans `.github/workflows/pipeline.yml`.

2 Notions CI/CD

Intégration continue (CI) : à chaque modification envoyée sur le dépôt, le code est récupéré, analysé (lint) et testé automatiquement sur une machine neutre (runner).

Livraison continue : le code validé par la CI est toujours prêt à être mis en production ; la mise en ligne peut rester déclenchée manuellement.

Déploiement continu : si tous les contrôles passent, la mise en ligne (ici simulée) s'exécute sans intervention.

La pipeline construite suit le schéma suivant : les jobs **Lint** et **Test** s'exécutent en parallèle ; **Build** ne démarre que si les deux réussissent ; **Deploy** ne démarre que si **Build** réussit et uniquement sur la branche `main` (condition `if: github.ref == 'refs/heads/main'`).

3 Étapes réalisées

3.1 Création du projet Python

J'ai créé le projet avec les fichiers suivants :

- `calculatrice.py` : fonctions `addition`, `soustraction`, `multiplication` et `division` ;
- `test_calculatrice.py` : cinq tests unitaires au départ, puis six après l'ajout de `puissance` ;
- `requirements.txt` : dépendance `pytest` ;
- `.gitignore` : exclusion de `venv/`, `__pycache__/` et `.pytest_cache/`.

La division par zéro lève une `ValueError`, ce qui est testé avec `pytest.raises`.

3.2 Tests et lint en local

En local, j'ai installé les dépendances, lancé les tests puis vérifié le style comme le fera la pipeline :

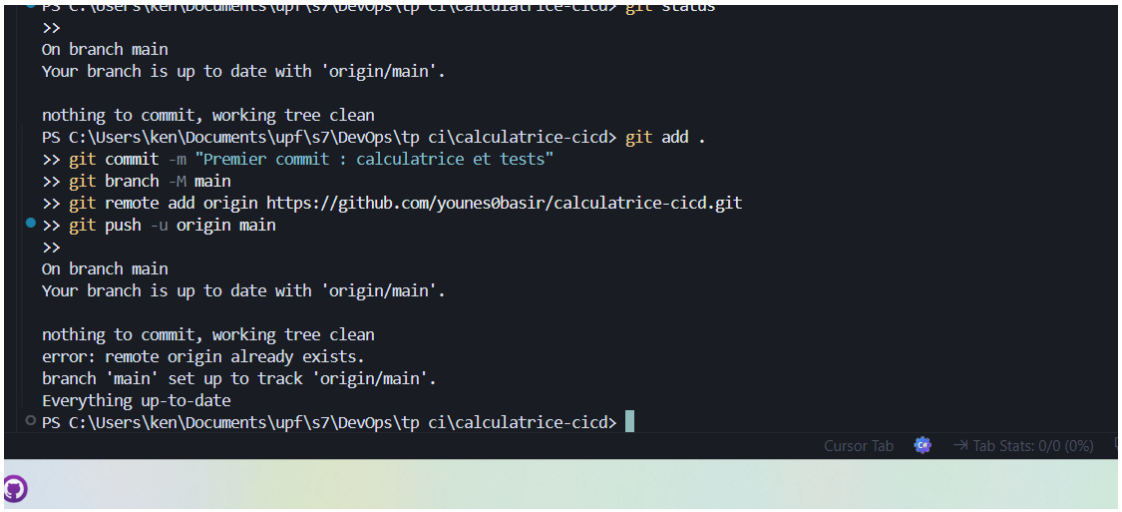


FIGURE 1 – Dépôt GitHub `calculatrice-cicd` avec les fichiers du premier commit.

```
python -m pip install -r requirements.txt
python -m pytest -v
python -m pip install flake8
python -m flake8 . --max-line-length=100
```

Cinq tests sont passés et `flake8` n'a signalé aucune erreur avant le premier envoi sur GitHub.

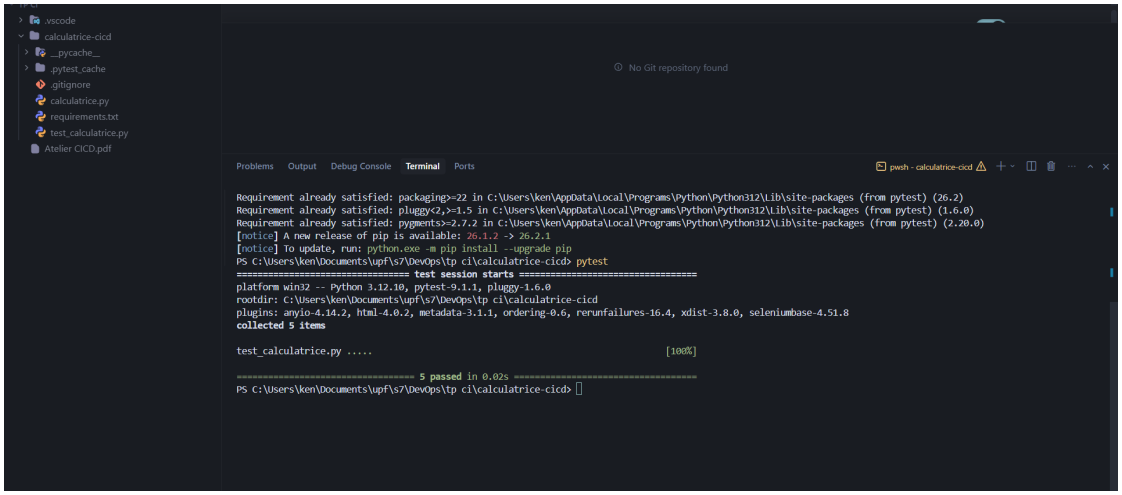


FIGURE 2 – Résultat de `pytest -v` en local (5 tests réussis).

3.3 Mise en place de GitHub Actions

J'ai ajouté le fichier `.github/workflows/pipeline.yml` (nom affiché : *Pipeline CI/CD Calculatrice*). La pipeline contient quatre jobs :

1. **Lint** : `flake8 . -max-line-length=100` sur Ubuntu, Python 3.12 ;
2. **Test** : `pytest -v` après installation de `requirements.txt` ;
3. **Build** : copie de `calculatrice.py` dans `dist/`, artefact `calculatrice-paquet` (`needs: [lint, test]`) ;
4. **Deploy** : téléchargement de l'artefact, `ls -l dist/`, message simulé (`needs: build`, condition `main` uniquement).

Les déclencheurs sont un `push` sur `main` et une `pull_request` vers `main`.

flake8.png

FIGURE 3 – Vérification du style avec `flake8` en local (aucune sortie = succès).

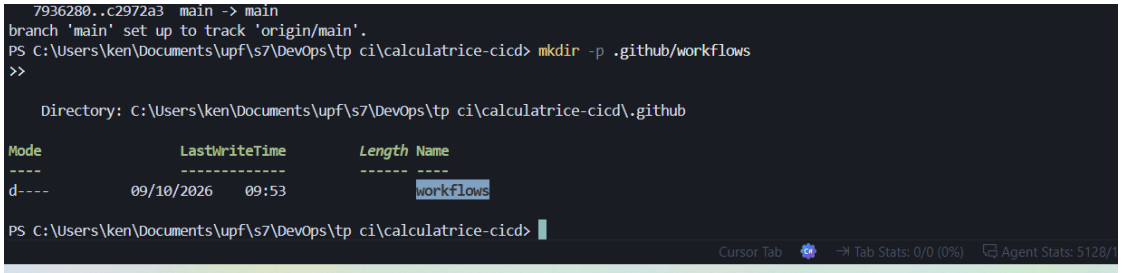


FIGURE 4 – Création du dossier `.github/workflows`.

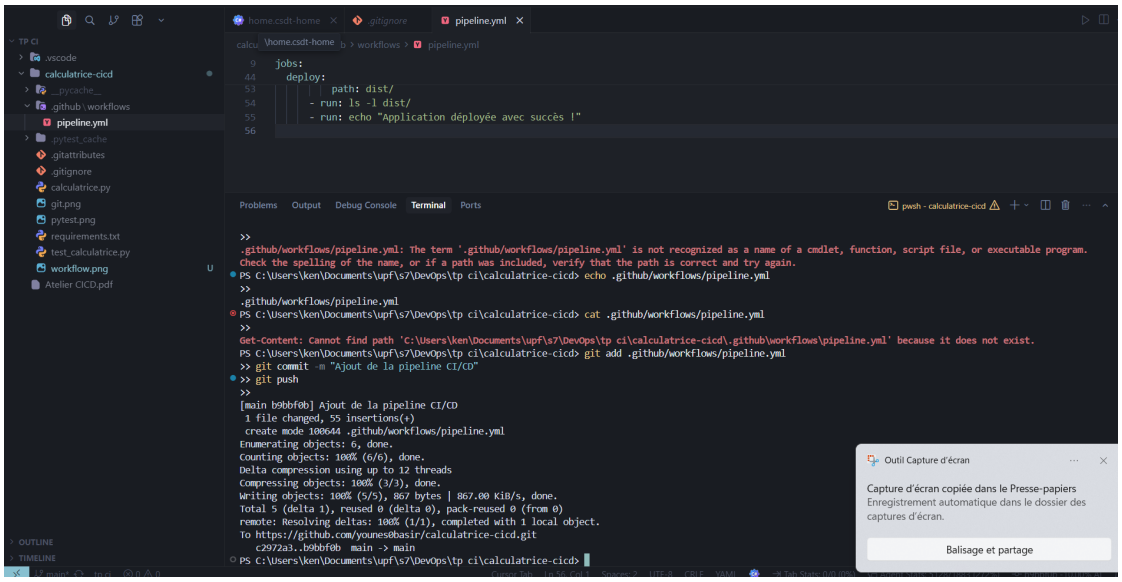


FIGURE 5 – Fichier `pipeline.yml` et envoi du workflow sur GitHub.

3.4 Observation de la pipeline (premier push réussi)

Après le `push` du workflow, j'ai ouvert l'onglet *Actions*, la dernière exécution, puis la page *Summary*. Le graphique montre **Lint** et **Test** côte à côte, puis **Build**, puis **Deploy**. J'ai consulté les logs du job *Tests automatiques* (les cinq tests visibles comme en local), téléchargé l'artefact `calculatrice-paquet` (archive contenant `calculatrice.py`) et lu le log du job *Déploiement (simulé)*.

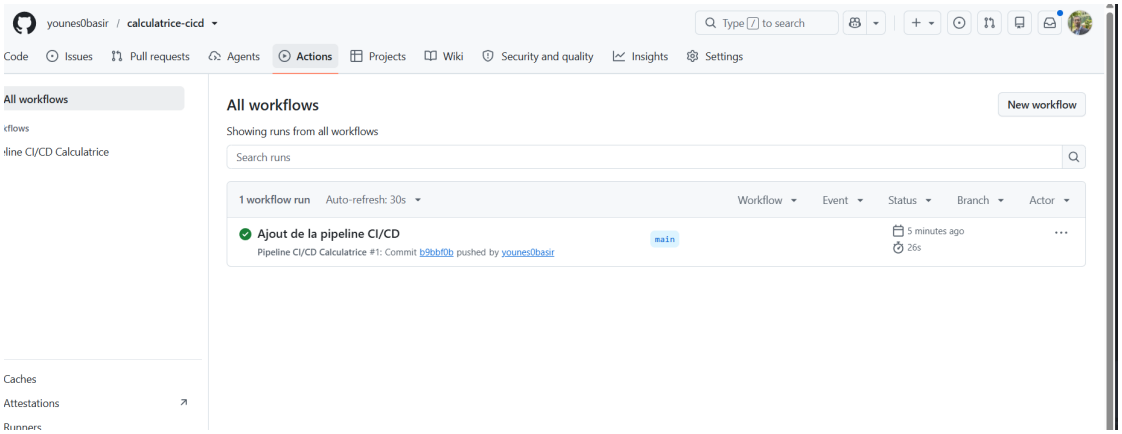


FIGURE 6 – Liste des exécutions ou vue d'ensemble dans l'onglet Actions.

3.5 Bug volontaire et correction (filet de sécurité)

Pour vérifier que la pipeline bloque un code incorrect, j'ai modifié `addition` en remplaçant `a + b` par `a - b`. En local, `pytest -v` a échoué sur `test_addition`. J'ai malgré tout poussé le commit (*Bug volontaire dans l'addition*).

Sur GitHub Actions, le job **Test** est passé au rouge. Les jobs **Build** et **Deploy** ne se sont pas exécutés (état gris / ignoré) : le code défectueux n'a donc pas été « déployé ». Après remise de `a + b`, vérification locale (`5 passed`) et push de correction, une nouvelle exécution est repassée entièrement au vert.

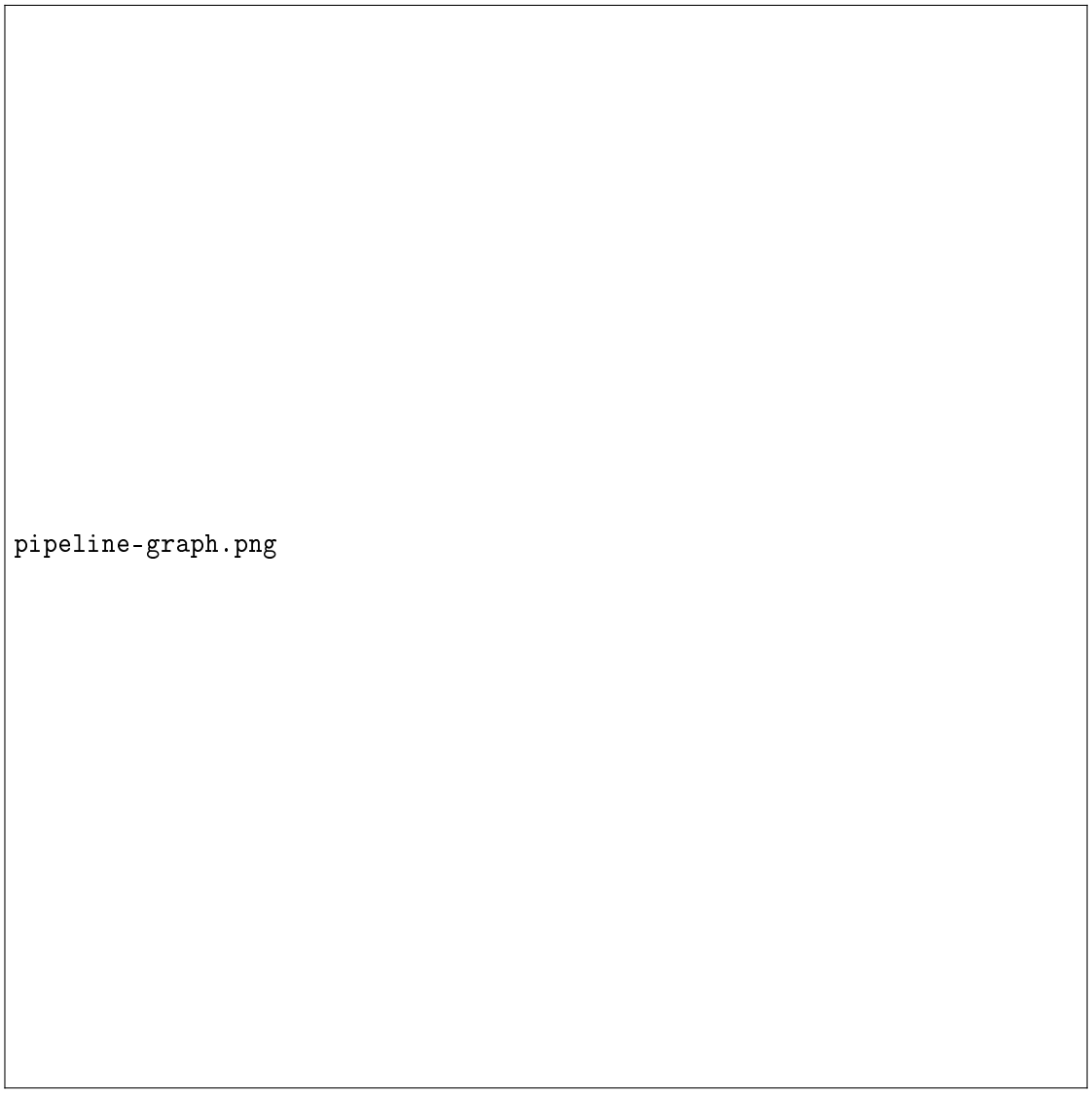


FIGURE 7 – Graphique des quatre jobs (Lint et Test en parallèle, puis Build et Deploy).

pytest-actions-log.png

FIGURE 8 – Logs du job *Tests automatiques* (`pytest -v`).

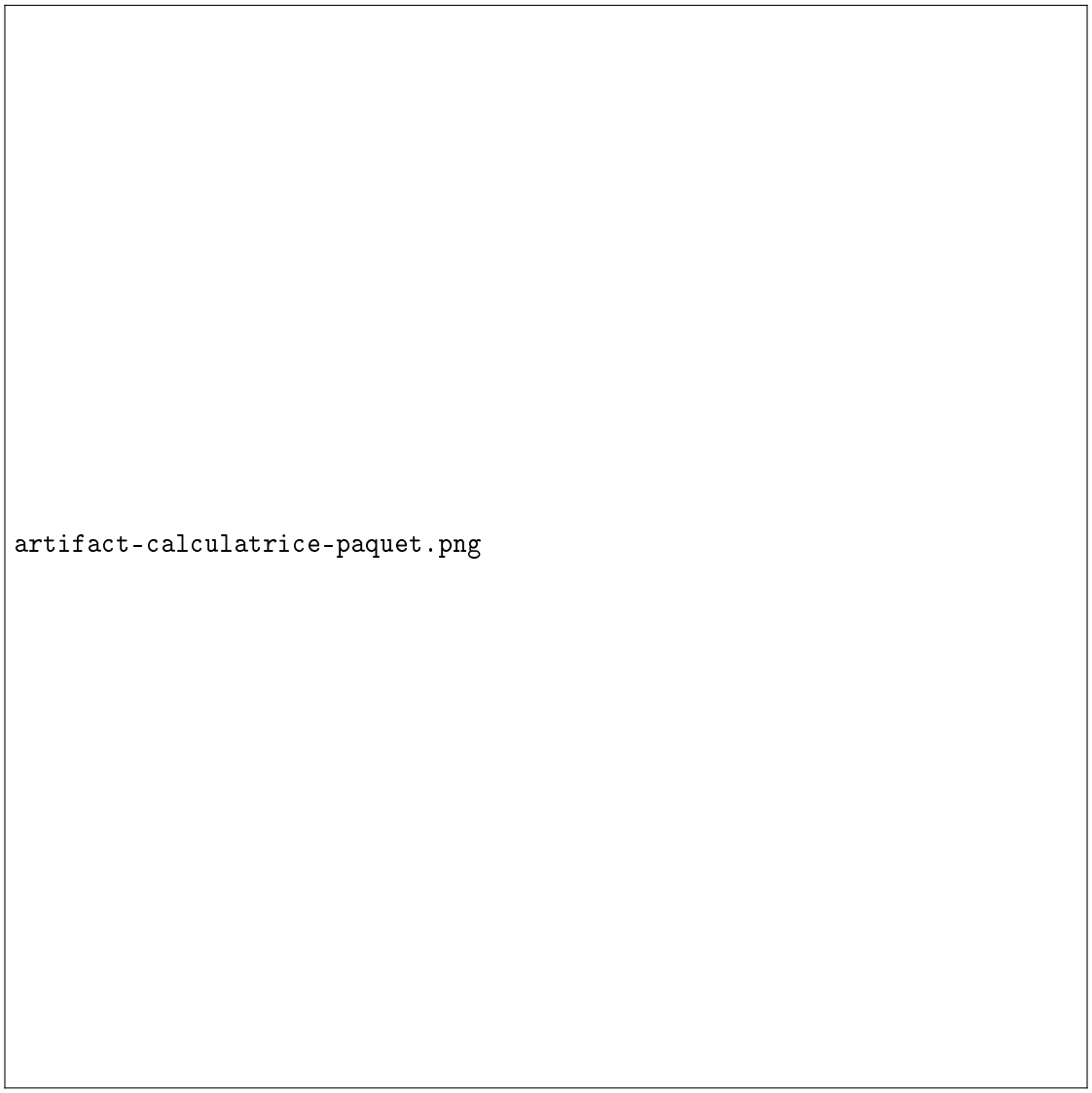


FIGURE 9 – Section *Artifacts* : téléchargement de `calculatrice-paquet`.

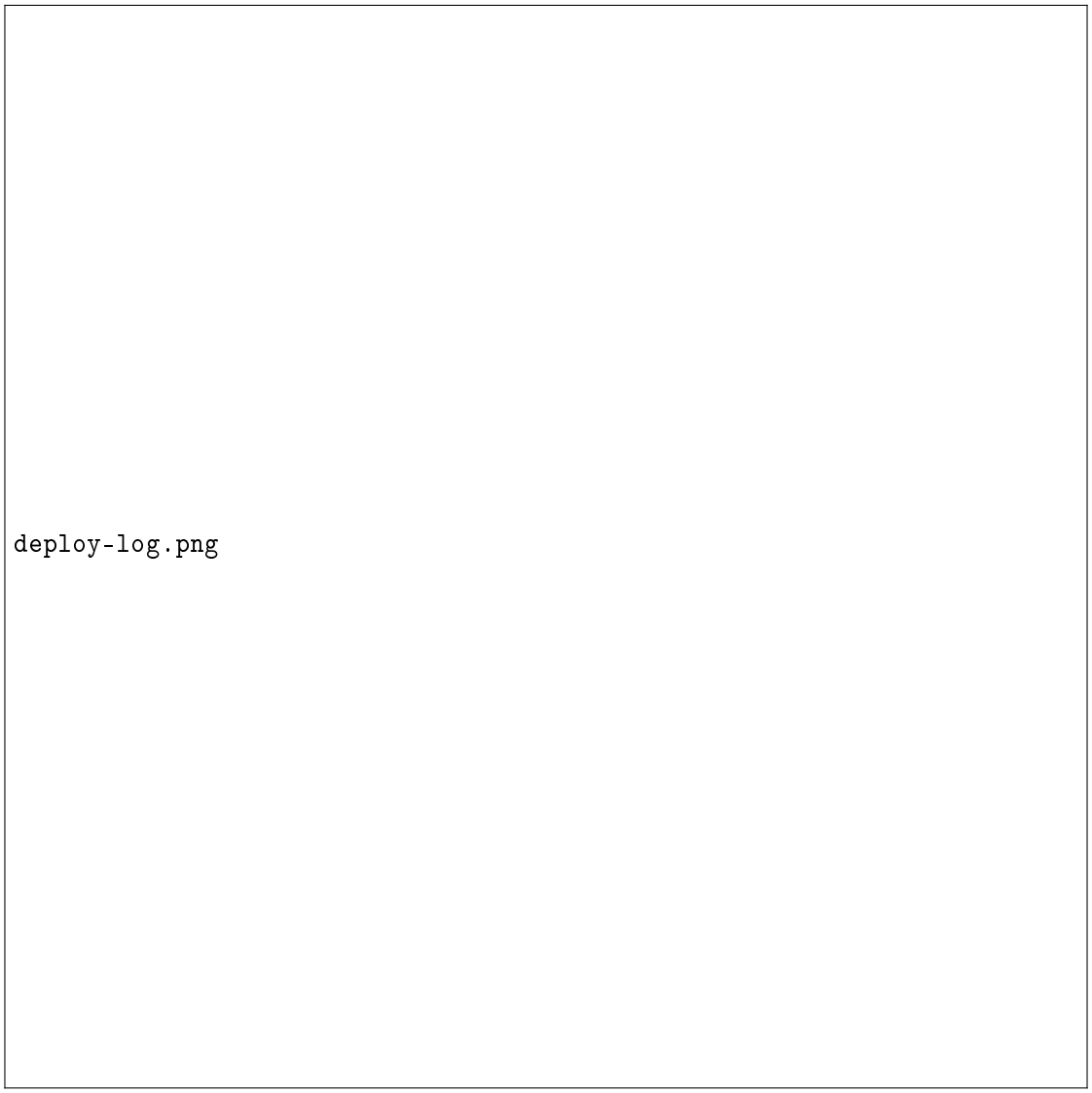


FIGURE 10 – Log du job *Déploiement (simulé)* (message de succès simulé).

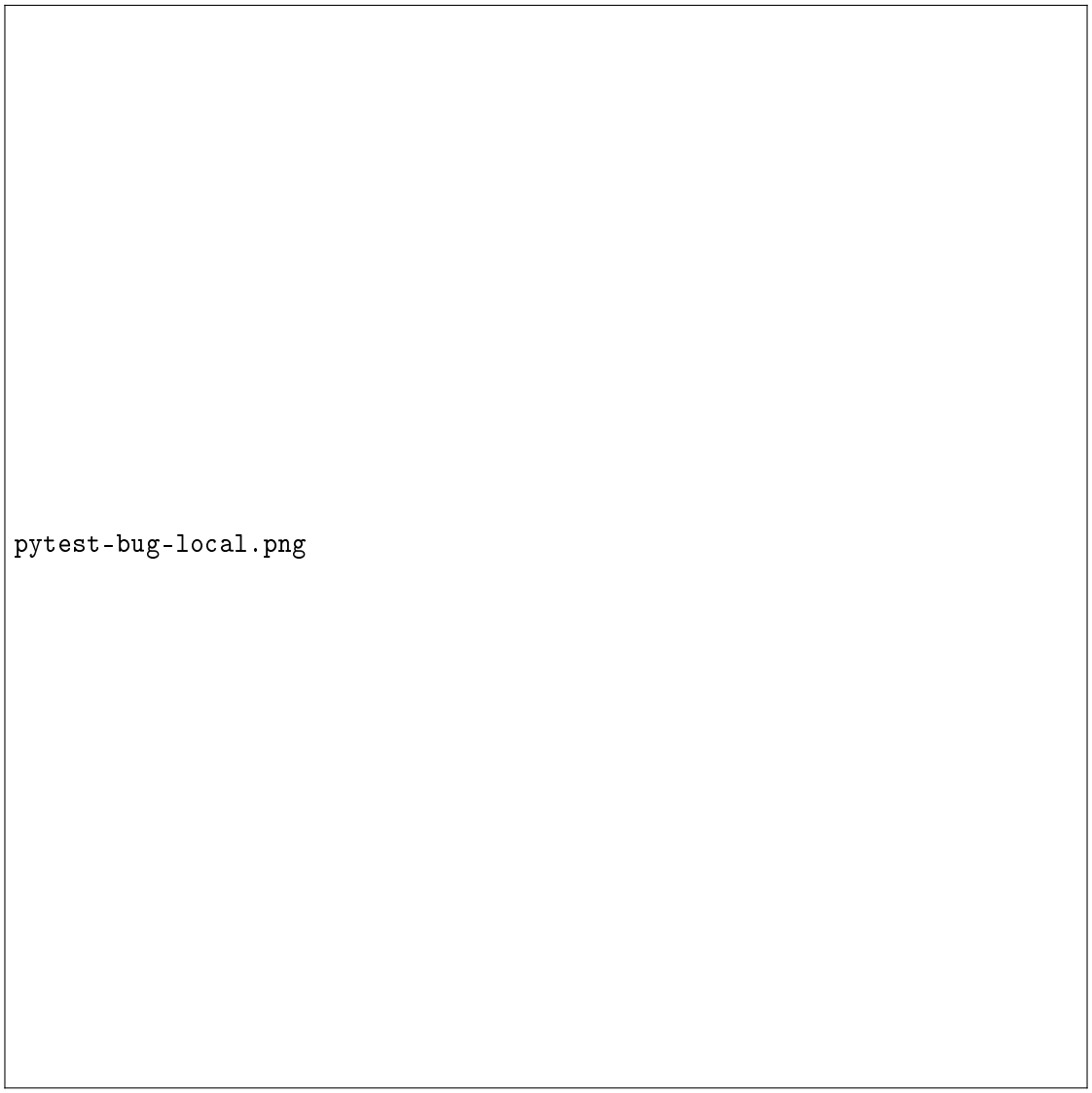


FIGURE 11 – `pytest` en local après introduction du bug (échec de `test_addition`).

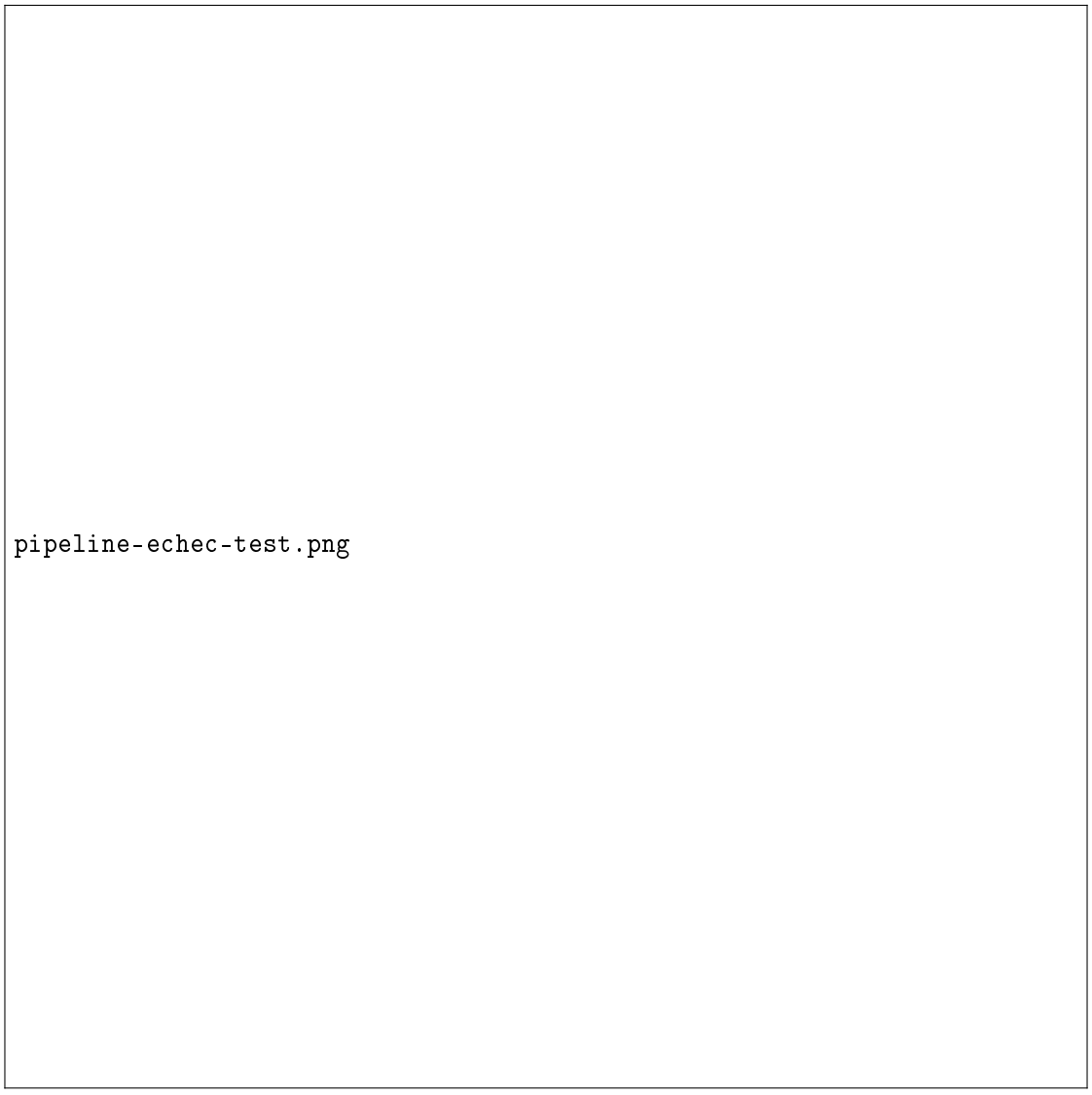


FIGURE 12 – Pipeline en échec : job *Tests automatiques* rouge, Build et Deploy non lancés.

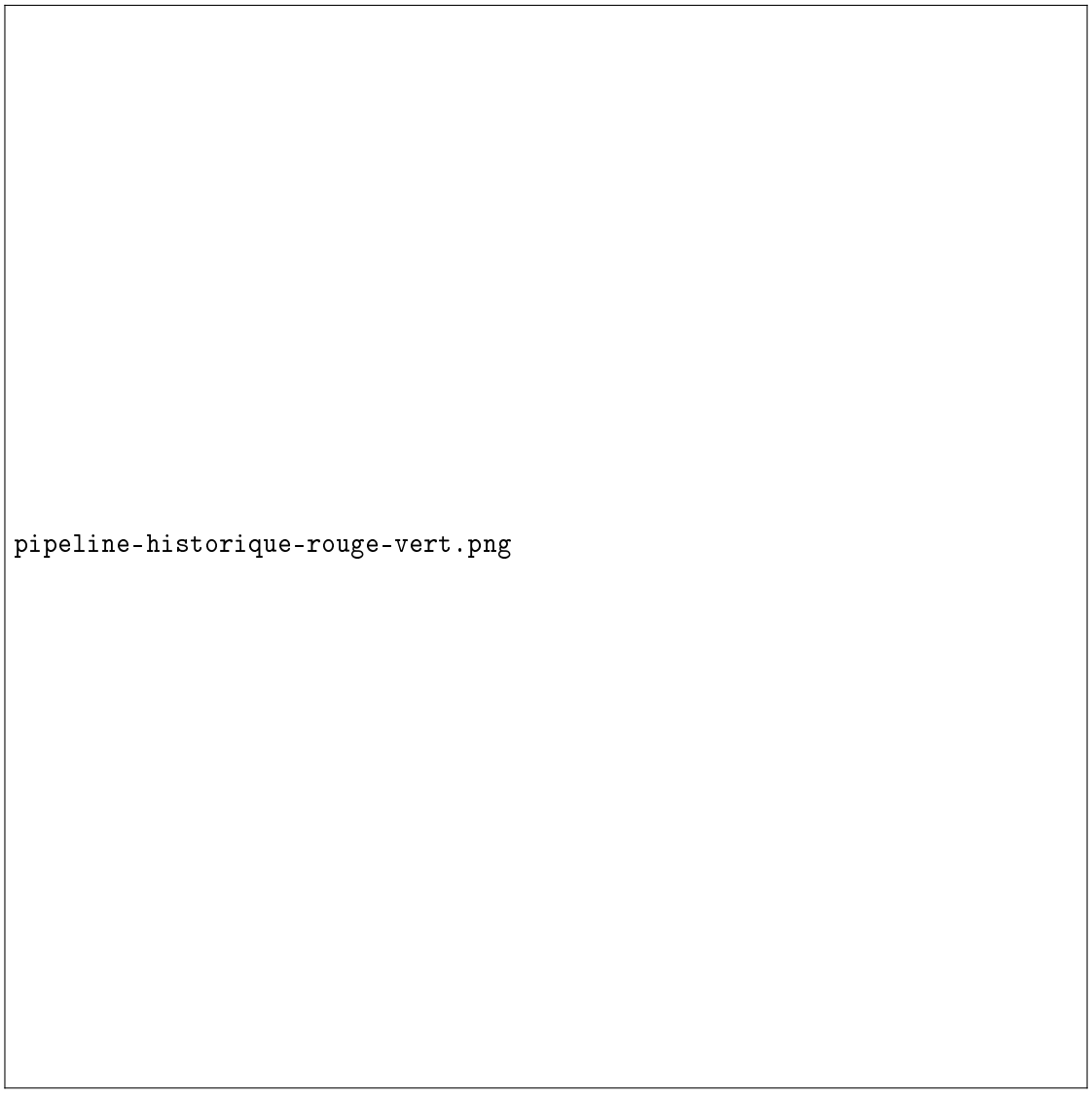


FIGURE 13 – Historique Actions : une exécution rouge puis une exécution verte après correction.

3.6 Branche `ajout-puissance` et Pull Request

Pour simuler un flux d'équipe, j'ai créé la branche `ajout-puissance`, ajouté `puissance(a, b)` dans `calculatrice.py` et `test_puissance` dans `test_calculatrice.py`, puis poussé la branche. En local, `pytest -v` affichait **6 passed**.

J'ai ouvert une **Pull Request** vers `main`. La pipeline s'est exécutée sur la PR : **Lint**, **Test** et **Build** ont réussi, mais **Deploy** est resté *skipped* (gris), car la condition `if` n'autorise le déploiement que depuis `main`, pas depuis une branche de PR.

Après *Merge pull request*, une nouvelle exécution sur `main` a cette fois exécuté **Deploy** avec succès. La branche `main` contient alors la calculatrice complète, y compris `puissance`.

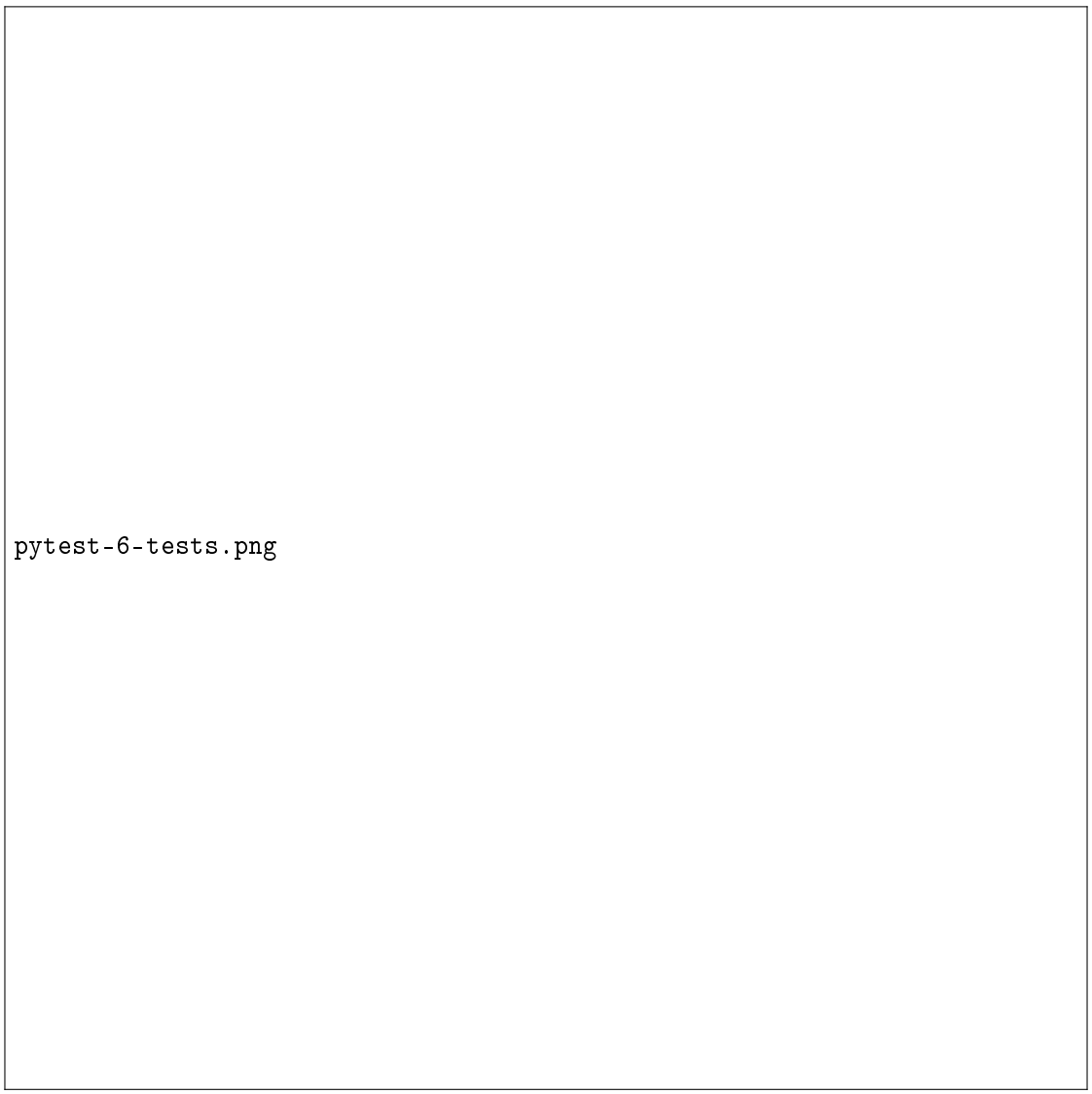


FIGURE 14 – `pytest -v` en local après ajout de `puissance` (6 tests).

4 Résultat final

Le projet dispose d'une pipeline à quatre jobs, déclenchée automatiquement sur `push` et `pull_request` vers `main`. Les tests et le lint garantissent la qualité ; un échec de test bloque build et déploiement. Le workflow branche → PR → merge → deploy sur `main` reproduit une pratique professionnelle de base. L'artefact `calculatrice-paquet` prouve que l'étape **Build** produit bien un livrable réutilisé par **Deploy**.

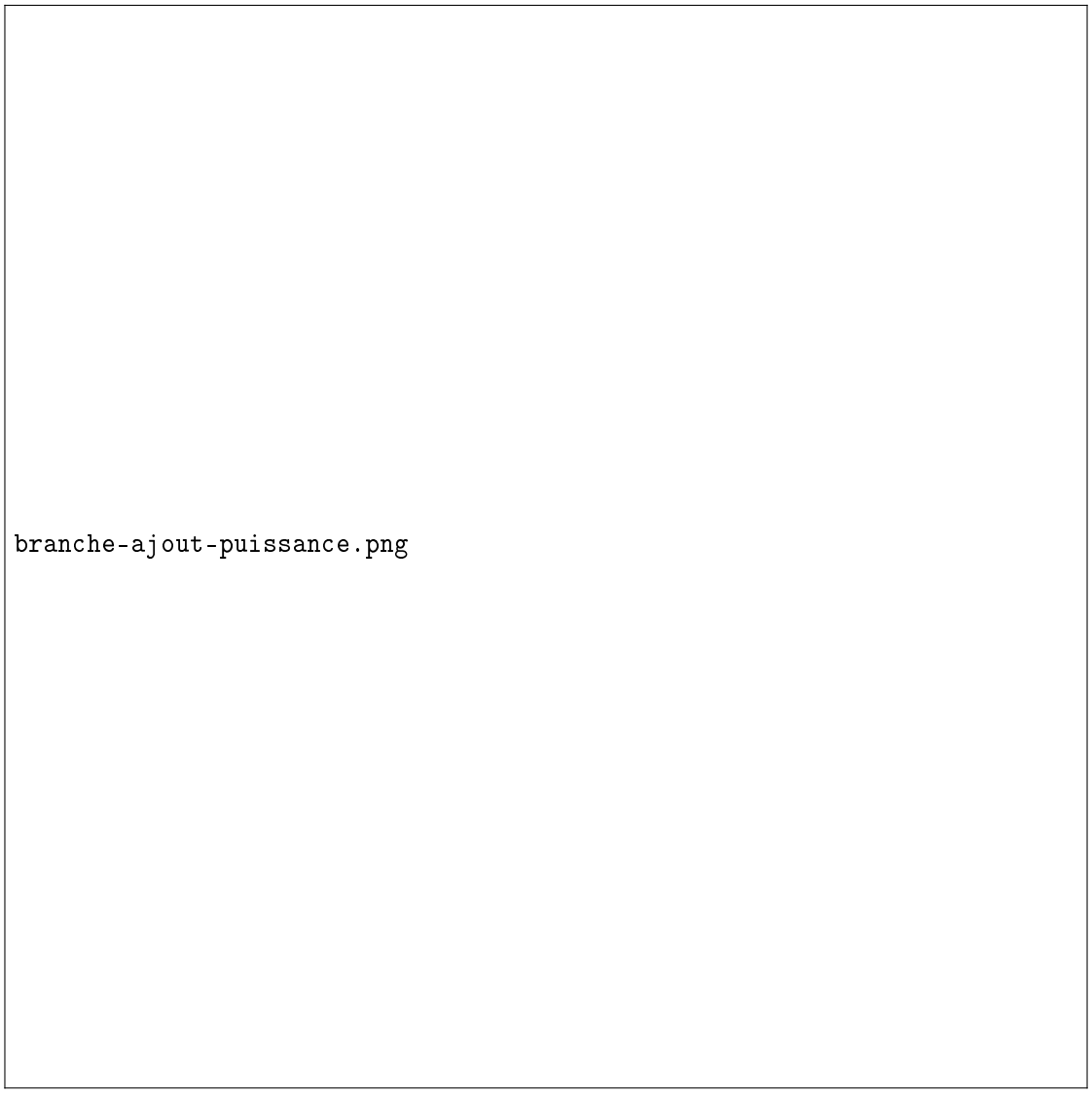


FIGURE 15 – Branche `ajout-puissance` sur GitHub (commits ou comparaison).

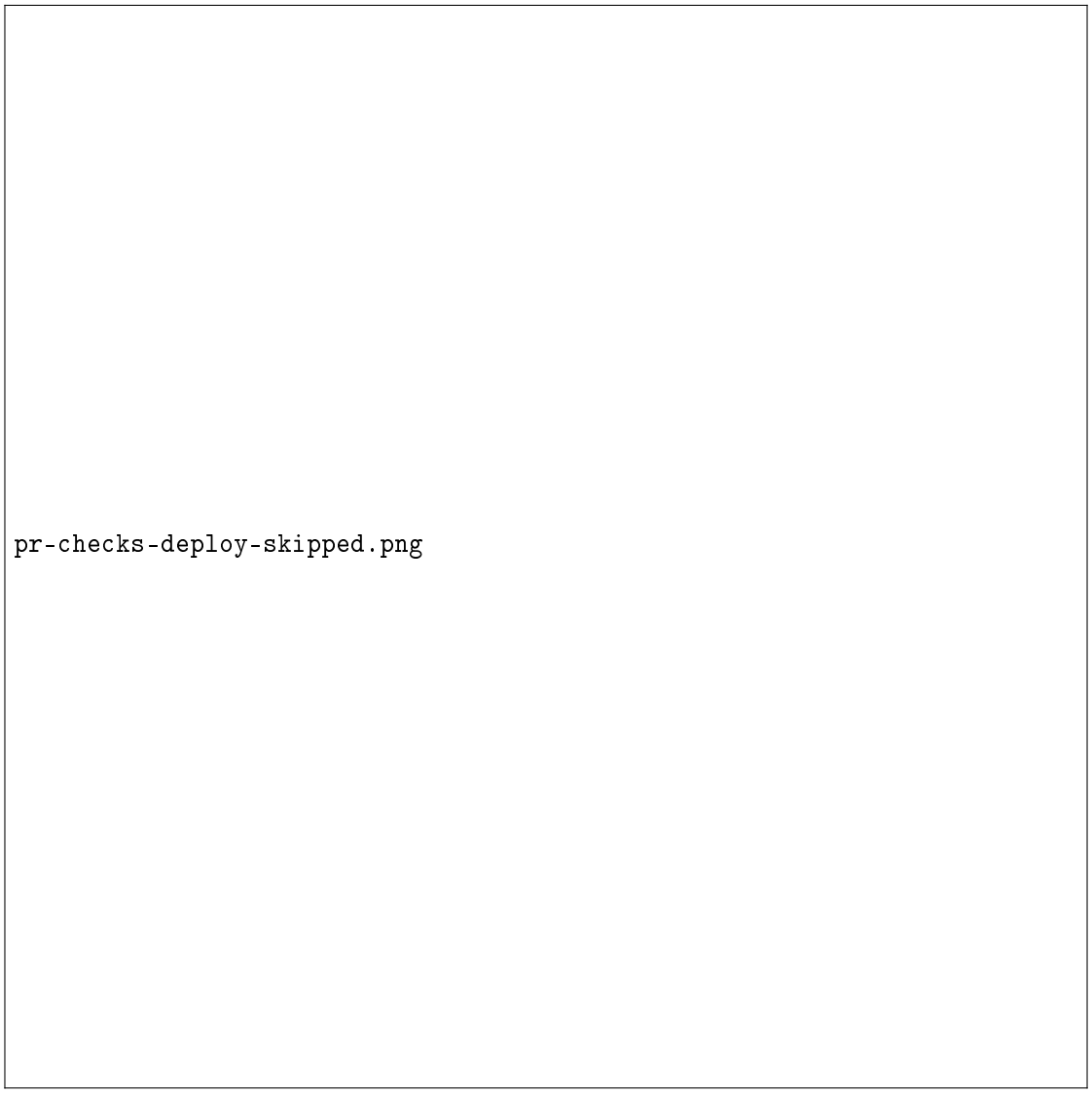


FIGURE 16 – Pull Request : checks CI verts et job *Deploy* ignoré (*skipped*).

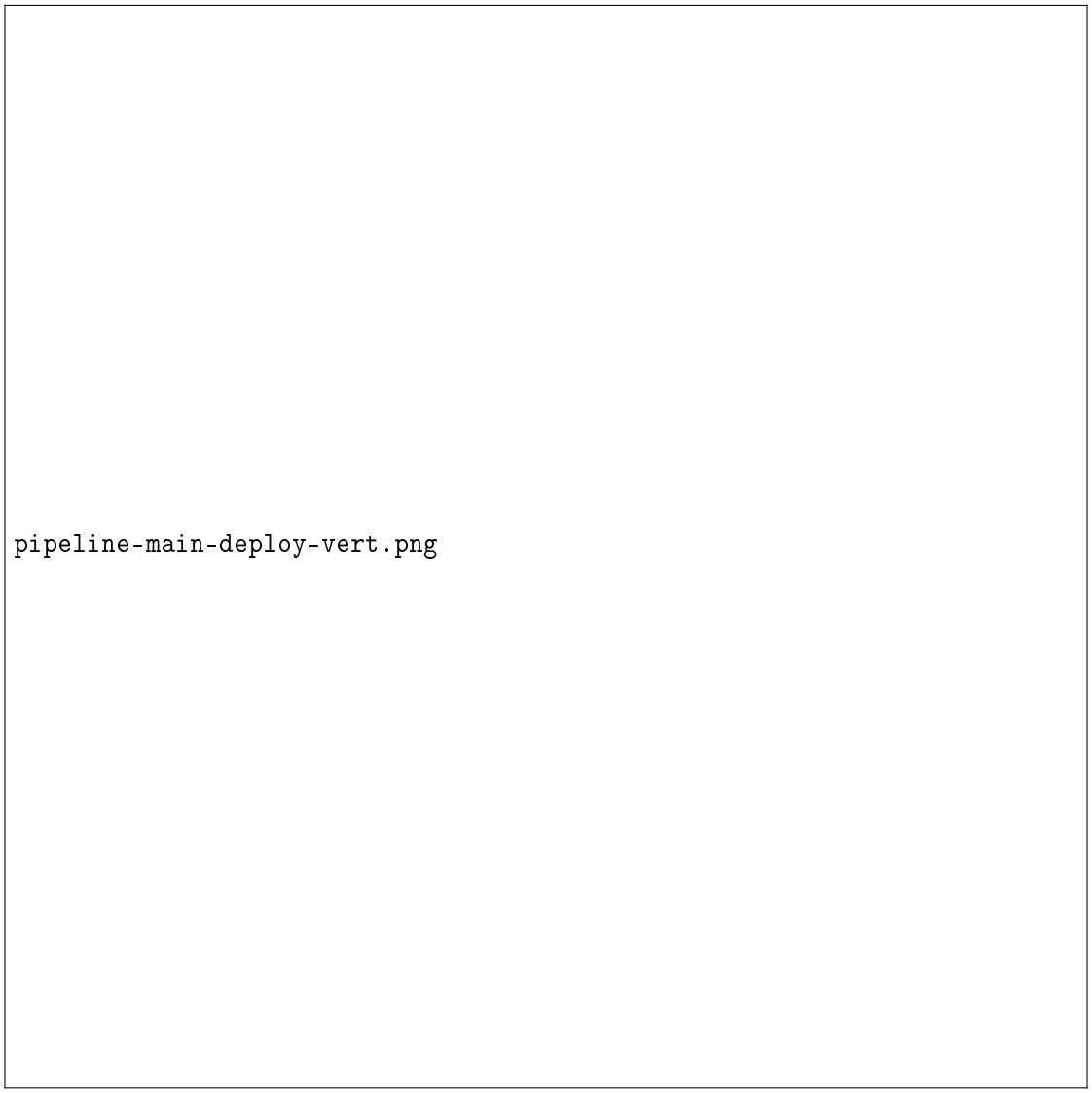


FIGURE 17 – Pipeline sur `main` après fusion : les quatre jobs verts, Deploy inclus.

5 Questions de réflexion

1. **CI vs déploiement continu :** la CI intègre et vérifie le code à chaque changement ; le déploiement continu publie automatiquement lorsque toutes les vérifications passent (ici le deploy est simulé et limité à `main`).
2. **Deploy gris quand Test est rouge : Build** dépend de **Test** via `needs: [lint, test]` ; sans build réussi, **Deploy** ne démarre pas.
3. **Lint et Test en parallèle :** les deux jobs sont indépendants ; les lancer ensemble réduit le temps total de la pipeline.
4. **Sans `if` sur Deploy :** le déploiement simulé s'exécuterait aussi sur les branches de PR, ce qui n'est en général pas souhaitable.
5. **Équipe de dix personnes :** même contrôle pour tous, traçabilité dans Actions, détection rapide des régressions, moins d'erreurs humaines ou de tests oubliés.

6 Conclusion

Cet atelier m'a permis de comprendre l'intérêt du CI/CD : les tests et le lint ne dépendent plus d'une vérification manuelle à chaque changement. GitHub Actions automatise ce contrôle à chaque `push` et chaque PR. La simulation d'un bug puis sa correction montrent le rôle de filet de sécurité de la pipeline. Pour la suite, on pourrait ajouter un badge de statut, `pytest-cov`, une image Docker dans **Build** ou un déploiement réel (GitHub Pages, Render, etc.).